Математика в профессии ИИ-специалиста

Линейная алгебра как фундамент ML

Глеб Карпов

ПриНТ ФКН ВШЭ

Матрица

- **Definition**

  Матрица — упорядоченный массив чисел в виде n строк и m столбцов.

$$A_{n\times m} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nm} \end{bmatrix}$$

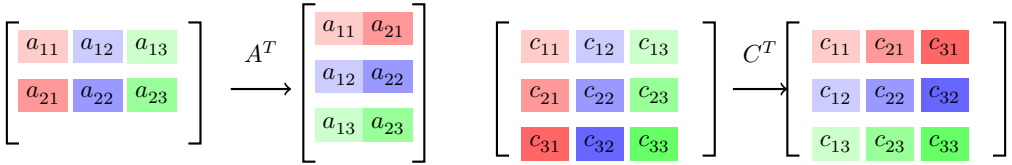
- Обычно обозначаем как $A_{n\times m}$ или, чтобы подчеркнуть природу чисел в матрице, пишем $A \in \mathbb{R}^{n\times m}$.
- Если $n = m$, то матрицу называют квадратной, если $n \neq m$ — прямоугольной

Основные операции

Транспонирование

Definition

Транспонирование матрицы — это операция, при которой строки и столбцы меняются местами. Если $A \in \mathbb{R}^{n \times m}$, то $B = A^T$, $B \in \mathbb{R}^{m \times n}$, где $b_{ij} = a_{ji}$



Основные операции

Сложение матриц

Definition

Сложение матриц возможно только для матриц одинакового размера. Результат получается сложением соответствующих элементов. Если $A, B \in \mathbb{R}^{n \times m}$, то $C = A + B$, где $c_{ij} = a_{ij} + b_{ij}$

Пример:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1+5 & 2+6 \\ 3+7 & 4+8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Основные операции

Умножение матрицы на скаляр

Definition

Умножение на скаляр — каждый элемент матрицы умножается на заданное число. Если $A \in \mathbb{R}^{n \times m}$ и $\alpha \in \mathbb{R}$, то $C = \alpha A$, где $c_{ij} = \alpha \cdot a_{ij}$

Пример:

$$3 \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 \cdot 1 & 3 \cdot 2 \\ 3 \cdot 3 & 3 \cdot 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

Комбинация операций:

$$2 \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + 3 \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix} = \begin{bmatrix} 5 & 3 \\ 3 & 5 \end{bmatrix}$$

Вектор

В самом простом представлении будем трактовать **вектор** как частный случай матрицы:

Вектор-столбец

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n \times 1}$$

Размерность: $n \times 1$ (матрица с одним столбцом)

Операции

Сложение: $x, y \in \mathbb{R}^n \Rightarrow z_i = x_i + y_i$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 5 \\ 7 \\ 9 \end{bmatrix}$$

Умножение на скаляр: $\alpha \in \mathbb{R} \Rightarrow y_i = \alpha \cdot x_i$

$$2 \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Обозначения

- **Векторы:** обычно обозначаются строчными буквами x, v
- **Матрицы:** обычно обозначаются заглавными буквами A, B, C
- **По умолчанию:** вектор считается **вектором-столбцом**

Умножение матрицы на вектор (matvec)

Обычный подход

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1,m-1} & a_{1m} \\ a_{21} & a_{22} & \cdots & a_{2,m-1} & a_{2m} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{n,m-1} & a_{nm} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{m-1} \\ x_m \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

A $\quad$ $\mathbf{x}$ $\quad$ $\mathbf{y}$

$$y_1 = a_{11}x_1 + a_{12}x_2 + \cdots + a_{1,m-1}x_{m-1} + a_{1m}x_m$$

$$\boxed{y_j = \sum_{k=1}^{m} a_{jk}x_k}$$

Умножение матрицы на вектор (matvec)

Вычислительная сложность и немного о параллельных вычислениях

Вспомним общую формулу:

$$y_j = \sum_{k=1}^{m} a_{jk} x_k, \quad j = 1, 2, \ldots, n$$

Анализ операций:

- Для вычисления одного элемента y_j: m умножений (каждое $a_{jk} \cdot x_k$), $m - 1$ сложений (суммирование m произведений)
- Для всего вектора $\mathbf{y}$ (n элементов): $n \cdot (2m - 1)$ операций всего.

Временная сложность:

- Для квадратной матрицы $n \times n$: $\mathcal{O}(2n^2 - n) = \mathcal{O}(n^2)$
- Для прямоугольной матрицы $n \times m$: $\mathcal{O}(nm)$

Умножение матрицы на вектор (matvec)

Вычислительная сложность и немного о параллельных вычислениях

Вспомним общую формулу:

$$y_j = \sum_{k=1}^{m} a_{jk} x_k, \quad j = 1, 2, \dots, n$$

Анализ операций:

- Для вычисления одного элемента y_j: m умножений (каждое $a_{jk} \cdot x_k$), $m - 1$ сложений (суммирование m произведений)
- Для всего вектора $\mathbf{y}$ (n элементов): $n \cdot (2m - 1)$ операций всего.

Временная сложность:

- Для квадратной матрицы $n \times n$: $\mathcal{O}(2n^2 - n) = \mathcal{O}(n^2)$
- Для прямоугольной матрицы $n \times m$: $\mathcal{O}(nm)$

Естественный параллелизм

Вычисление каждого элемента y_j **независимо** от других элементов!
Построчная параллелизация: отдельный процессор может вычислять свой y_j

Умножение матрицы на вектор (matvec)

Просветленный подход :)

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}}_{\mathbf{y}}$$

Умножение матрицы на вектор (matvec)

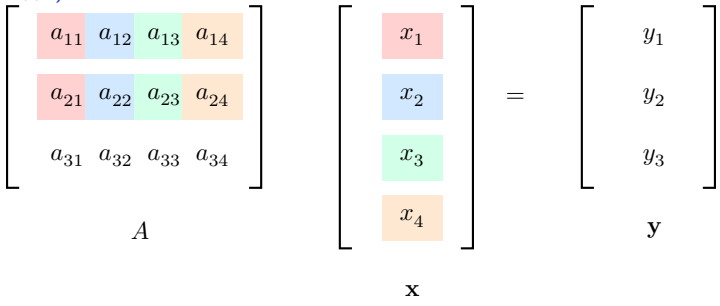
Просветленный подход :)

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}}_{\mathbf{y}}$$

$$y_1 = a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4$$

Умножение матрицы на вектор (matvec)

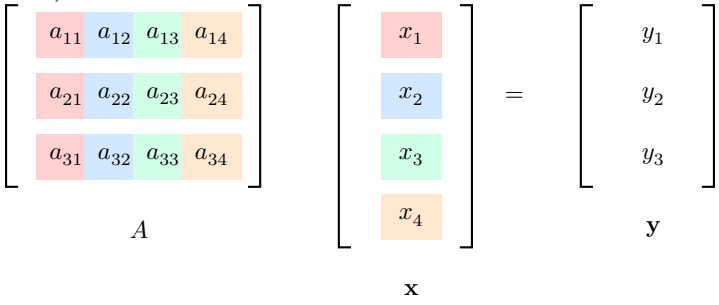
Просветленный подход :)



$$y_1 = a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4$$
$$y_2 = a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4$$

Умножение матрицы на вектор (matvec)

Просветленный подход :)



$$y_1 = a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + a_{14}x_4$$
$$y_2 = a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + a_{24}x_4$$
$$y_3 = a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + a_{34}x_4$$

Умножение матрицы на вектор (matvec)

Просветленный подход :)

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}}_{\mathbf{y}}$$

$$\mathbf{y} = x_1 \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix} + x_2 \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \end{bmatrix} + x_3 \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \end{bmatrix} + x_4 \begin{bmatrix} a_{14} \\ a_{24} \\ a_{34} \end{bmatrix}$$

Умножение матрицы на вектор

Визуальное сравнение

$$\begin{bmatrix} 1 & 1 \\ 1 & 4 \end{bmatrix} \quad \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$$

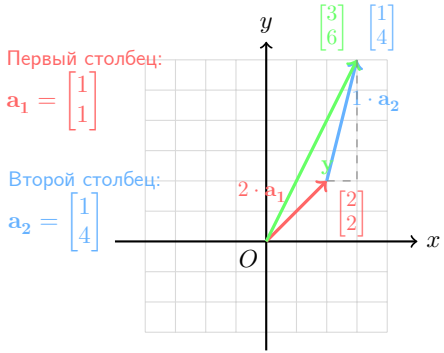
A $\quad$ $\mathbf{x}$ $\quad$ $\mathbf{y}$

Строка на столбец

Вычисления:

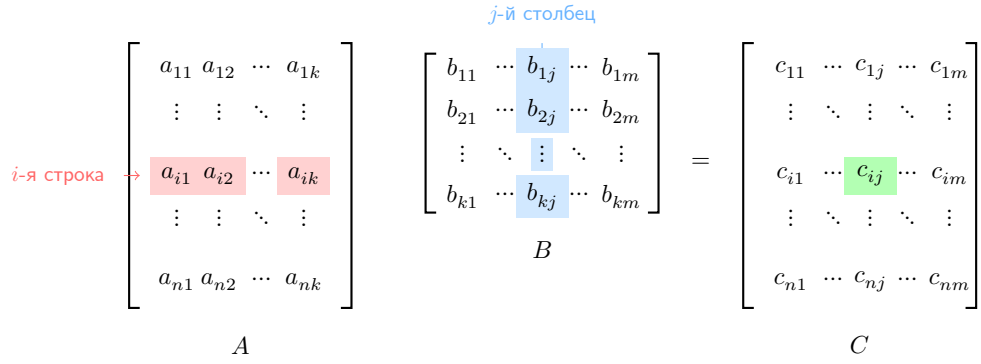
- $y_1 = 1 \cdot 2 + 1 \cdot 1 = 3$
- $y_2 = 1 \cdot 2 + 4 \cdot 1 = 6$

Комбинация столбцов



Умножение матриц (matmul, GEneral Matrix Multiplication - GEMM)

Обычный подход



$$c_{ij} = a_{i1} \cdot b_{1j} + a_{i2} \cdot b_{2j} + \cdots + a_{ik} \cdot b_{kj}$$

$$\boxed{c_{ij} = \sum_{l=1}^{k} a_{il} b_{lj}}$$

Умножение матриц (matmul, GEneral Matrix Multiplication - GEMM)

Столбцовый подход

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{ik} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nk} \end{bmatrix} \quad A$$

$$\begin{bmatrix} b_{11} & b_{12} & b_{13} & \cdots & b_{1m} \\ b_{21} & b_{22} & b_{23} & \cdots & b_{2m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{k1} & b_{k2} & b_{k3} & \cdots & b_{km} \end{bmatrix} \quad B$$

$$= \quad \begin{bmatrix} c_{11} & c_{12} & c_{13} & \cdots & c_{1m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{i1} & c_{i2} & c_{i3} & \cdots & c_{im} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & c_{n3} & \cdots & c_{nm} \end{bmatrix} \quad C$$

Каждый столбец $C = A\times$ соответствующий столбец B

$$\boxed{\mathbf{C_j} = A\mathbf{B_j}, \quad j = 1, 2, \ldots, m}$$

Умножение матриц

Вычислительная сложность и параллелизация

Вспомним общую формулу:

$$c_{ij} = \sum_{l=1}^{k} a_{il} b_{lj}, \quad i = 1, 2, \ldots, n, \quad j = 1, 2, \ldots, m$$

Анализ операций для матриц $A_{n \times k}$ и $B_{k \times m}$:

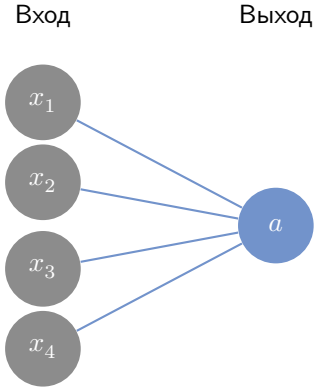
- Для вычисления одного элемента c_{ij}: k умножений, $k - 1$ сложений
- Общее количество операций: $n \times m \times (2k - 1)$
- Для квадратных матриц $n \times n$ количество операций пропорционально n^3: сложность $\mathcal{O}(n^3)$
- Для прямоугольных матриц: сложность $\mathcal{O}(nmk)$.

Естественный параллелизм

- **По элементам:** каждый c_{ij} вычисляется независимо
- **По столбцам:** каждый процессор обрабатывает свои столбцы матрицы B

Перцептрон

Один нейрон: 4 входа $\rightarrow$ 1 выход



Функции активации σ

ReLU: $\sigma(z) = \max(0,\, z)$

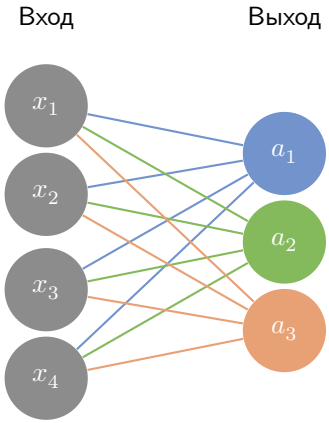
Sigmoid: $\sigma(z) = \dfrac{1}{1 + e^{-z}}$

tanh: $\sigma(z) = \tanh(z)$

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b, \qquad a = \sigma(z)$$

Перцептрон

Один слой: несколько нейронов



Каждый нейрон — свой набор весов. Цвет связи = строка матрицы W.

Полносвязный слой

Матричная запись: один слой = matvec

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}$$

Например, $z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + b_1$ — тот же matvec, что на первом слайде

Полносвязный слой

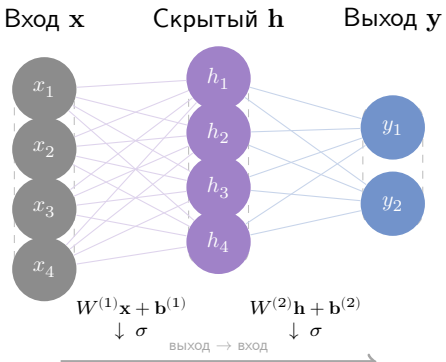
Активация поэлементно

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix} \xrightarrow{\sigma} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \qquad \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \sigma(z_1) \\ \sigma(z_2) \\ \sigma(z_3) \end{bmatrix}$$

Итого: $\mathbf{a} = \sigma(W\mathbf{x} + \mathbf{b})$ — вся операция слоя сводится к matvec + нелинейность

MLP — многослойный перцептрон

Слои соединяются последовательно



Каждый слой — matvec + σ. Выход $\mathbf{h}$ — вход для $W^{(2)}$.

Размеры

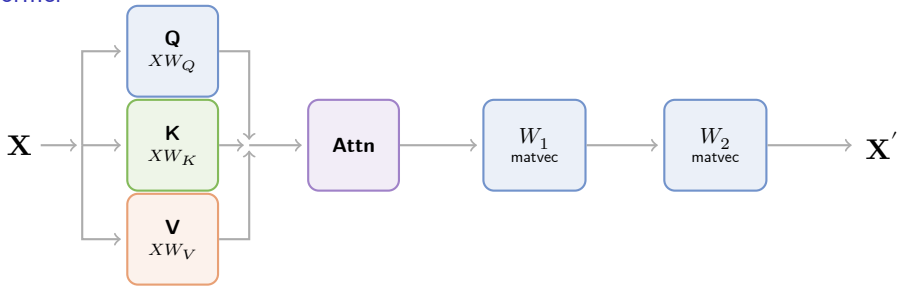
$\mathbf{x} \in \mathbb{R}^4$

$W^{(1)} \in \mathbb{R}^{4\times 4}$
$\mathbf{b}^{(1)} \in \mathbb{R}^4$
$\Rightarrow \mathbf{h} \in \mathbb{R}^4$

$W^{(2)} \in \mathbb{R}^{2\times 4}$
$\mathbf{b}^{(2)} \in \mathbb{R}^2$
$\Rightarrow \mathbf{y} \in \mathbb{R}^2$

LLM — в сути те же matvec

Блок Transformer



$\times N$ таких блоков подряд $\Rightarrow$ GPT, LLaMA, ...

$\mathbf{X}W_Q$, $\mathbf{X}W_K$, $\mathbf{X}W_V$ — три matmul с одним входом

$\mathbf{H}W_1$, $\mathbf{H}W_2$ — два полносвязных слоя, как в MLP

Пример: GPT-3 (175B) — 96 Transformer блоков

- $W_Q, W_K, W_V \in \mathbb{R}^{12\,288 \times 12\,288}$
- $W_1 \in \mathbb{R}^{12\,288 \times 49\,152}$, $\quad W_2 \in \mathbb{R}^{49\,152 \times 12\,288}$
- $\Rightarrow$ 175 млрд параметров — сумма размеров всех таких матриц по всем слоям

За пределами matvec

Линейная алгебра — не только умножения

Мы посмотрели: нейросети и LLM на 99% — это **matvec** и **matmul**.

Но линал даёт и другие инструменты:

- **сжатие** данных и матриц;
- **выделение** главной структуры в данных.

Проблема: в GPT-3 одна матрица весов $12\,288 \times 12\,288$ — это уже ~ 150 млн чисел.

Всего **175 млрд** параметров — как это хранить и передавать?

Как запускать такие огромные модели, например, на телефоне или другом вычислительно слабом устройстве?

Может, большую матрицу можно записать **компактнее**?

SVD — сингулярное разложение

Любую матрицу можно «разложить на множители»

Definition

SVD: для любой матрицы A существует разложение

$$A = U \Sigma V^{\mathsf{T}}$$

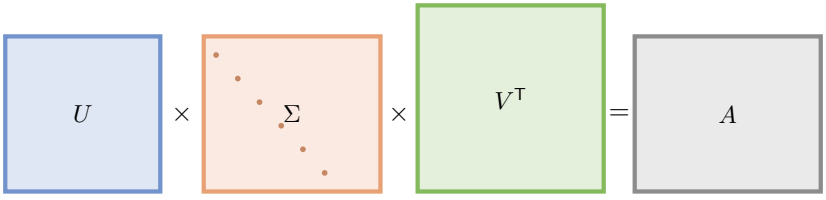
- U, V^{T} — «вспомогательные» матрицы для настройки преобразования (переход между базисами)
- Σ — **диагональная** матрица с числами $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ на диагонали

Эти σ_i называются **сингулярными значениями** — чем больше σ_i, тем «важнее» соответствующее направление в данных.

SVD — сингулярное разложение

Три матрицы вместо одной

$$A = U \Sigma V^{\mathsf{T}}$$



Точки на диагонали Σ — сингулярные значения σ_i.

SVD — зачем это в ML?

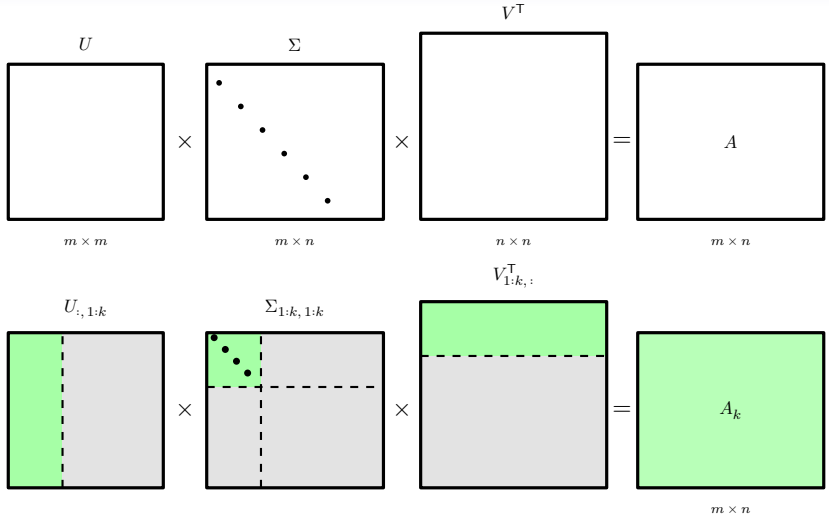
Аккуратно убираем ненужную информацию из матрицы

Часто $\sigma_1, \sigma_2, \ldots, \sigma_k$ уже несут почти всю информацию — остальные малы.
Усечение ранга k: берём только первые k столбцов U, k значений Σ и k строк V^T.

Матрица A_k получается **меньше** по объёму хранения, но близка к исходной A.

Применения: сжатие изображений, уменьшение размерности данных, **приближение больших матриц весов** (low-rank approximation).

Полное и сжатое SVD: блоки $U\,\Sigma\,V^{\mathsf{T}}$



Вверху — полное произведение $U\Sigma V^{\mathsf{T}} = A$ без усечения. Ниже — те же три фактора, но с выделением блоков, из которых собирается $A_k = U_{:,\,1:k}\,\Sigma_{1:k,\,1:k}\,V^{\mathsf{T}}_{1:k,\,:}$: зелёным — входящие в произведение для ранга не выше k, серым — остальные части полной записи.